プログラミングワークショップ２

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

今後の予定

回	内容
~~1~~	~~ソフトウェア開発1~~
~~2~~	~~ソフトウェア開発2~~
~~3~~	~~ソフトウェア開発3~~
~~4~~	~~ソフトウェア開発4~~
~~5~~	~~中間発表1(テーマ発表)~~
~~6~~	~~ソフトウェア開発5~~
~~7~~	~~ソフトウェア開発6~~
~~8~~	~~ソフトウェア開発7~~

回	内容
~~9~~	~~ソフトウェア開発8~~
~~10~~	~~中間発表2(開発状況)~~
~~11~~	~~中間発表2(開発状況)つづき~~
~~12~~	~~ソフトウェア開発9~~
~~13~~	~~ソフトウェア開発10~~
14	ソフトウェア開発11(補講)
15	最終発表(補講)

- ソフトウェア開発：12回
- 中間発表：2回（テーマ発表、開発状況）
- 最終発表：1回

最終発表

- 2/6(火)4時間目
- 中間発表2からの作業状況
- プログラムのデモ
  - 実装した機能の実演
  - 状態遷移図・表の変更

メインタブ・Buttonクラス①

```
//メインタブ sample04 (変更なし)

Calc calc;

void setup() {
  size(400, 475);
  calc = new Calc();
}

void draw() {
  calc.run();
}
```

```
//Buttonクラス (変更なし)
class Button {

  int x, y, w, h;
  String name;
  boolean wasPressed;
  int buttonColor = color(255);
  int hoverColor  = color(0, 255, 255);
  int pushColor   = color(255, 255, 0);
  int textColor   = color(0);

  Button(int _x, int _y, int _w, int _h,
                        String _name) {
    x = _x;
    y = _y;
    w = _w;
    h = _h;
    name = _name;
  }
```

Buttonクラス②

```
//カーソルがボタン上にあるかどうか
 boolean isHover() {
   if ((mouseX > x) && (mouseX < x+w) &&
     (mouseY > y) && (mouseY < y+h)) {
     return true;
   } else {
     return false;
   }
 }

 //ボタンを押したかどうか
 boolean isPressed() {
   if (isHover() && mousePressed) {
     return true;
   } else {
     return false;
   }
 }
```

```
//ボタンをクリックしたかどうか
 boolean isClicked() {
   boolean isPressed = isPressed();
   boolean res;

   if (!wasPressed && isPressed) {
     res = true;
   } else {
     res = false;
   }
   wasPressed = isPressed;
   return res;
 }

 //ボタンの名前を返す
 String getName() {
   return name;
 }
```

Buttonクラス③

```
 //ボタンを表示する
  void display() {
    if (isPressed()) {
      fill(pushColor);
    } else if (isHover()) {
      fill(hoverColor);
    } else {
      fill(buttonColor);
    }
    rect(x, y, w, h);
    textAlign(CENTER, CENTER);
    textSize(32);
    fill(textColor);
    text(name, x, y, w, h);
  }

}
```

Screenクラスの変更

- 画面に表示する
  - 画面に文字列を表示する

    `void display(String s)`
  - 画面に実数を表示する

    `void display(float f)`

どちらも同じ
メソッド名

- メソッドのオーバーロード
  - 引数の型、数の異なる同じ名前のメソッドを複数定義できる

Screenクラス①

```
//Screenクラス

class Screen {

  int x, y, w, h;  //x座標、y座標、幅、高さ
  int screenColor = color(0);    //画面の色
  int textColor   = color(255);  //文字の色

  //コンストラクタ
  Screen(int _x, int _y, int _w, int _h){
    x = _x;
    y = _y;
    w = _w;
    h = _h;
  }
```

Screenクラス②

```
//画面に文字列を表示する
  void display(String s){
    fill(screenColor);  //画面の色を指定
    rect(x, y, w, h);   //長方形を描く

    textAlign(RIGHT, CENTER);  //文字の配置を右寄せ、上下中央寄せに指定
    textSize(48);              //テキストサイズ指定
    fill(textColor);           //テキストの色を指定
    text(s, x+10, y+10, w-20, h-20);  //画面外枠の10pt内側に文字を書く
  }

  //画面に実数を表示する
  void display(float f){
    display(str(f));     //実数sを文字列に変換し画面に表示する
  }

}
```

Calcクラス変更(前回分)

- フィールド追加
  - 数値1　　float num1;
  - 数値2　　float num2;
  - 入力途中の数値
    String input;
  - 演算方法　String ope;
  - 答え　　float answer;
  - 現在の状態
    State state;

- メソッド追加・変更
  - 電卓を表示する（変更）
    void display(String str)
  - 実行する
    void run()
  - 押したボタンの名前を返す
    String getButtonName()

Calcクラス変更(今回)

- メソッド追加・変更
  - 電卓を表示する `void display(float f)`
  - 計算する `float calc(float n1, float n2)`
  - 連結する `String concat(String a, string b)`
  - 数字ボタンを押したかどうか
    `boolean isClickedNumButton(String b)`
  - 演算ボタンを押したかどうか
    `boolean isClickedOpeButton(String b)`
  - ＝ボタンを押したかどうか
    `boolean isClickedEqButton(String b)`
  - Cボタンを押したかどうか
    `boolean isClickedClearButton(String b)`

計算する

- 数値num1,num2と演算方法opeで四則演算
  float calc(float n1, float n2)
  - opeが”+”の場合は、n1+n2を返す
  - opeが”-”の場合は、n1-n2を返す
  - opeが”*”の場合は、n1*n2を返す
  - opeが”/”の場合は、n1/n2を返す

連結する

- 入力途中の数値inputに押したボタンの数値bを連結する
  - ただし、2つ目の小数点は無視
  - 連結した文字列を実数に変換し非実数(NaN)でない場合、連結した文字列を返す
  - それ以外は連結前の文字列を返す
- 文字列aと文字列bを連結、返す

```
String concat(String a, String b)
```

非数 NaN（Not a Number）

- 文字列を浮動小数点数に変換

  float(文字列)

  - 文字列が数値に変換不可能な場合 NaN

- 判定

  - NaNかどうか Float.isNaN(数)
    - true　非数 NaN
    - False　非数でない

Calcクラス①

```
//状態
enum State {
    INIT,        //初期状態
    INPUT_NUM1,  //数値1入力中
    SELECT_OP,   //演算入力中
    INPUT_NUM2,  //数値2入力中
    RESULT       //結果表示中
}
//Calcクラス
class Calc {
  Screen screen; //画面
  Button[] btn;  //ボタン(配列)
  float num1;    //数値1
  float num2;    //数値2
  String input;  //入力途中の数値
  String ope;    //演算方法
  float answer;  //答え
  State state;   //現在の状態
```

Calcクラス②

```
//コンストラクタ
Calc() {
  num1 = 0.0;
  num2 = 0.0;
  answer = 0.0;
  input = "0";
  ope = "";
  state = State.INIT;

  screen = new Screen(0, 0, 400, 100);
  btn = new Button[17];
  btn[0]  = new Button(0, 400, 200, 75, "0");
  btn[1]  = new Button(0, 325, 100, 75, "1");
  btn[2]  = new Button(100, 325, 100, 75, "2");
  btn[3]  = new Button(200, 325, 100, 75, "3");
  btn[4]  = new Button(0, 250, 100, 75, "4");
  btn[5]  = new Button(100, 250, 100, 75, "5");
  btn[6]  = new Button(200, 250, 100, 75, "6");
```

Calcクラス③

```
    btn[7]  = new Button(0, 175, 100, 75, "7");
    btn[8]  = new Button(100, 175, 100, 75, "8");
    btn[9]  = new Button(200, 175, 100, 75, "9");
    btn[10] = new Button(200, 400, 100, 75, ".");
    btn[11] = new Button(300, 325, 100, 150, "=");
    btn[12] = new Button(300, 175, 100, 150, "+");
    btn[13] = new Button(300, 100, 100, 75, "-");
    btn[14] = new Button(200, 100, 100, 75, "*");
    btn[15] = new Button(100, 100, 100, 75, "/");
    btn[16] = new Button(0, 100, 100, 75, "C");
  }

  //電卓を表示する
  void display(String str) {
    //画面を表示する
    screen.display(str);
    //ボタンを表示する
    for (int i=0; i<btn.length; i++) {
```

Calcクラス④

```
    btn[i].display();
  }
}

//電卓を表示する
void display(float f) {
  screen.display(f);  //画面に実数を表示する
  //ボタンを表示する
  for (int i=0; i<btn.length; i++) {
    btn[i].display();
  }
}

//押したボタンの名前を返す
String getButtonName() {
  String res = "";
  for (int i=0; i<btn.length; i++) { //すべてのボタンで繰り返す
    if (btn[i].isClicked()) {  //i番目のボタンをクリックしたら
```

Calcクラス⑤

```
        res = btn[i].getName();  //そのボタンの名前を代入する
        break;                   //繰り返しから抜ける
      }
    }
    return res;
  }

  //数字ボタンを押したかどうか
  boolean isClickedNumButton(String s) {
    if (s == "0" || s == "1" || s == "2" || s == "3" ||
      s == "4" || s == "5" || s == "6" || s == "7" ||
      s == "8" || s == "9" || s == ".") {
      return true;
    } else {
      return false;
    }
  }
```

小数点(.)も含める

Calcクラス⑥

```
//演算ボタンを押したかどうか
boolean isClickedOpeButton(String s) {
  if (s == "+" || s == "-" || s == "*" || s == "/") {
    return true;
  } else {
    return false;
  }
}

//=ボタンを押したかどうか
boolean isClickedEqButton(String s) {
  if (s == "=") {
    return true;
  } else {
    return false;
  }
}
```

Calcクラス⑦

```
//Cボタンを押したかどうか
boolean isClickedClearButton(String s) {
  if (s == "C") {
    return true;
  } else {
    return false;
  }
}

//計算する
float calc(float n1, float n2) {
  if (ope == "+") {
    return n1 + n2;
  } else if (ope == "-") {
    return n1 - n2;
  } else if (ope == "*") {
    return n1 * n2;
  } else if (ope == "/") {
```

Calcクラス⑧

```
    return n1 / n2;
  } else {
    return 0;
  }
}

//連結する
String concat(String a, String b) {

  String ret = a + b;

  //連結した文字列を数値に変換してNaNになる場合
  if (Float.isNaN(float(ret))) {
    return a;    //連結前の文字列を返す
  } else {
    return ret;  //連結した文字列を返す
  }
}
```

Calcクラス⑨

```
  //押したボタンの名前を返す
  String getButtonName() {
    String res = "";
    for (int i=0; i<btn.length; i++) { //すべてのボタンで繰り返す
      if (btn[i].isClicked()) {  //i番目のボタンをクリックしたら
        res = btn[i].getName();  //そのボタンの名前を代入する
        break;                   //繰り返しから抜ける
      }
    }
    return res;
  }

//実行する
  void run() {
    String b = getButtonName();  //押したボタンの名前を変数bに代入
    switch(state) {
      //初期状態の場合
    case INIT:
```

Calcクラス⑨

```
//必ず実行する処理
num1 = 0.0;          //数値１を初期化
num2 = 0.0;          //数値２を初期化
input = "0";         //入力途中の数値を初期化
ope = "";            //演算方法をを初期化
display(input);  //入力途中の数値を画面に表示

//イベントごとの処理
if (isClickedNumButton(b)) { //数字ボタンを押した場合
  input = concat(input, b);  //押した数値を入力途中の数値に連結する
  state = State.INPUT_NUM1;  //現在の状態を数値1入力中に遷移
} else if (isClickedOpeButton(b)) {  //演算ボタンを押した場合
  num1 = 0.0;  //数値1に0を代入
  ope = b;     //演算方法に入力した演算を代入
  state = State.SELECT_OP;  //現在の状態を演算入力中に遷移
} else if (isClickedEqButton(b)) {  //=ボタンを押した場合
  answer = 0.0;          //答えに0を代入
  state = State.RESULT;  //現在の状態を結果表示中に遷移
```

Calcクラス⑩

```
    } else if (isClickedClearButton(b)) {  //Cボタンを押した場合
      state = State.INIT;  //現在の状態を初期状態に遷移
    }
    break;

    //数値1入力中の場合
  case INPUT_NUM1:
    //必ず実行する処理
    display(float(input));  //入力途中の数値を実数に変換して表示する
    //イベントごとの処理
    if (isClickedNumButton(b)) { //数字ボタンを押した場合
      input = concat(input, b);  //押した数値を入力途中の数値に連結する
      state = State.INPUT_NUM1;  //現在の状態を数値1入力中に遷移
    } else if (isClickedOpeButton(b)) {  //演算ボタンを押した場合
      num1 = float(input);       //数値1に入力途中の数値を代入
      ope = b;                   //演算方法に入力した演算を代入
      state = State.SELECT_OP;  //現在の状態を演算入力中に遷移
    } else if (isClickedEqButton(b)) {  //=ボタンを押した場合
```

Calcクラス⑪

```
      //無視
    } else if (isClickedClearButton(b)) {  //Cボタンを押した場合
      state = State.INIT;  //現在の状態を初期状態に遷移
    }
    break;

  //演算入力中の場合
  case SELECT_OP:
    //必ず実行する処理
    display(num1);  //数値1を表示する
    //イベントごとの処理
    if (isClickedNumButton(b)) {  //数字ボタンを押した場合
      input = b;                  //押した数値を入力途中の数値に代入する
      state = State.INPUT_NUM2;    //数値2入力中に遷移
    } else if (isClickedOpeButton(b)) { //演算ボタンを押した場合
      ope = b;                          //演算方法に入力した演算を代入
      state = State.SELECT_OP;          //演算入力中に遷移
    } else if (isClickedEqButton(b)) {  //=ボタンを押した場合
```

Calcクラス⑫

```
    //無視
  } else if (isClickedClearButton(b)) {  //Cボタンを押した場合
    state = State.INIT;  //初期状態に遷移
  }
  break;

//数値2入力中の場合
case INPUT_NUM2:
  //必ず実行する処理
  display(float(input));  //入力途中の数値を実数に変換して表示する
  //イベントごとの処理
  if (isClickedNumButton(b)) {   //数字ボタンを押した場合
    input = concat(input, b);    //押した数値を入力途中の数値に連結する
    state = State.INPUT_NUM2;    //数値2入力中に遷移
  } else if (isClickedOpeButton(b)) {  //演算ボタンを押した場合
    //無視
  } else if (isClickedEqButton(b)) { //=ボタンを押した場合
    num2 = float(input);             //数値2に入力途中の数値を代入
```

Calcクラス⑬

```
    answer = calc(num1, num2);        //演算実行
    state = State.RESULT;             //結果表示中に遷移
  } else if (isClickedClearButton(b)) {  //Cボタンを押した場合
    state = State.INIT;  //初期状態に遷移
  }
  break;

//結果表示中の場合
case RESULT:
  //必ず実行する処理
  display(answer);  //答えを表示する
  //イベントごとの処理
  if (isClickedNumButton(b)) {  //数字ボタンを押した場合
    //無視
  } else if (isClickedOpeButton(b)) {  //演算ボタンを押した場合
    //無視
  } else if (isClickedEqButton(b)) {  //=ボタンを押した場合
    //無視
```

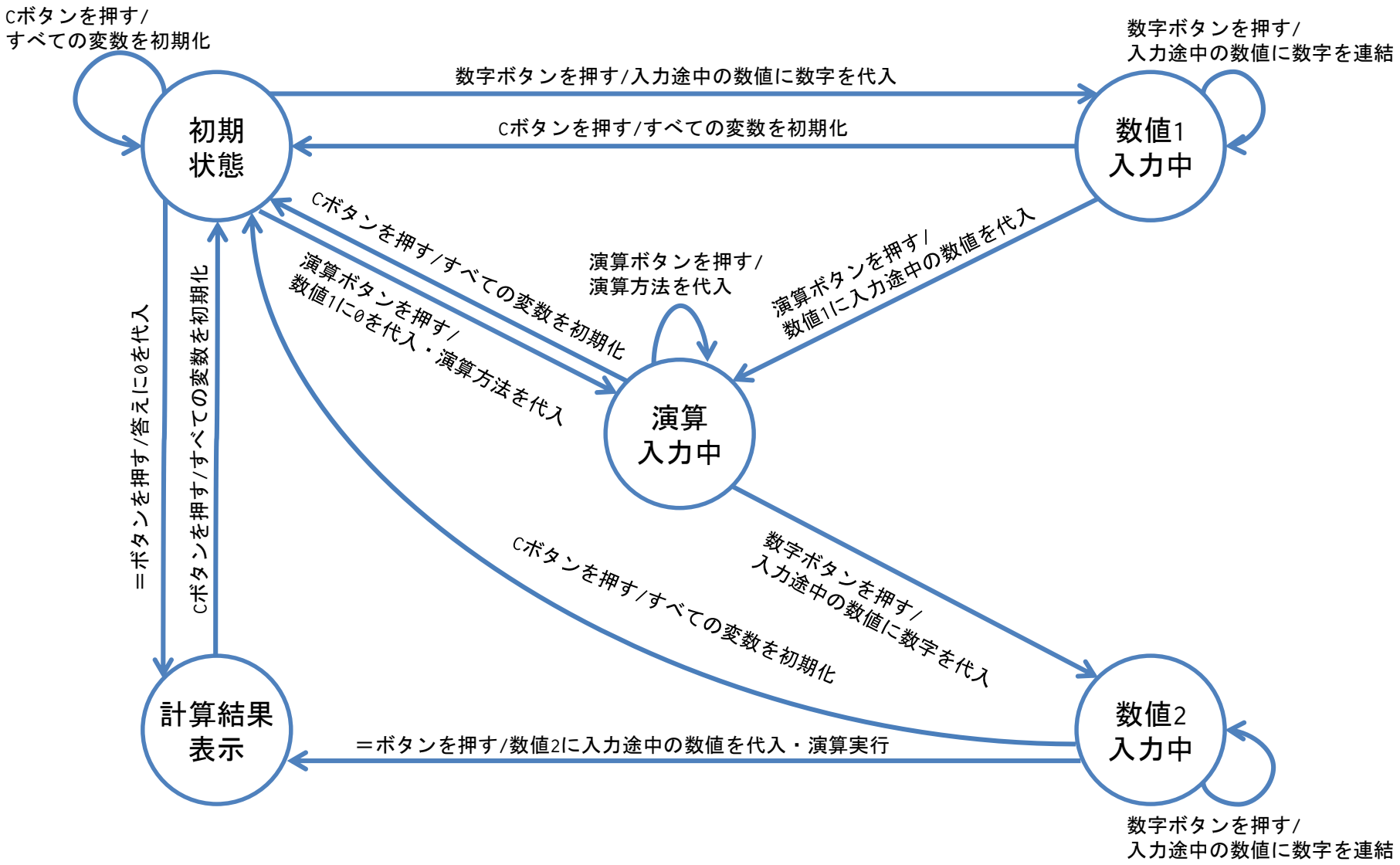
Calcクラス⑭

```
    } else if (isClickedClearButton(b)) {  //Cボタンを押した場合
      state = State.INIT;  //初期状態に遷移
    }
    break;

    //その他
  default:
    break;
  }
}

}
```

状態遷移表

	初期状態	数値1 入力中	演算 入力中	数値2 入力中	結果 表示中
数字ボタンを押す	入力途中の数値に数字を代入	入力途中の数値に数字を連結	入力途中の数値に数字を代入	入力途中の数値に数字を連結	?
	数値1入力中	数値1入力中	数値2入力中	数値2入力中	?
演算ボタンを押す	数値1に0を代入 演算方法を代入	数値1に 入力途中の数値を代入	演算方法を代入	?	?
	演算入力中	演算入力中	演算入力中	?	?
＝ボタンを押す	答えに0を代入	?	?	数値2に入力途中の数値を代入・演算実行	?
	結果表示中	?	?	結果表示中	?
Cボタンを押す	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化
	初期状態	初期状態	初期状態	初期状態	初期状態

状態遷移図


Cボタンを押す/
すべての変数を初期化
初期
状態
数字ボタンを押す/入力途中の数値に数字を代入
Cボタンを押す/すべての変数を初期化
数字ボタンを押す/
入力途中の数値に数字を連結
数値1
入力中
Cボタンを押す/すべての変数を初期化
演算ボタンを押す/
演算方法を代入
演算ボタンを押す/
数値1に入力途中の数値を代入
演算ボタンを押す/
数値1に0を代入・演算方法を代入
演算
入力中
=ボタンを押す/答えに0を代入
Cボタンを押す/すべての変数を初期化
Cボタンを押す/すべての変数を初期化
数字ボタンを押す/
入力途中の数値に数字を代入
計算結果
表示
=ボタンを押す/数値2に入力途中の数値を代入・演算実行
数値2
入力中
数字ボタンを押す/
入力途中の数値に数字を連結